

UNIVERSIDAD DE MANIZALES
FACULTAD DE CIENCIAS E INGENIERÍA

SEGURIDAD EN DESARROLLO DE SOFTWARE

PRESENTADO POR:

Jorge Mauricio Jurado Moreno

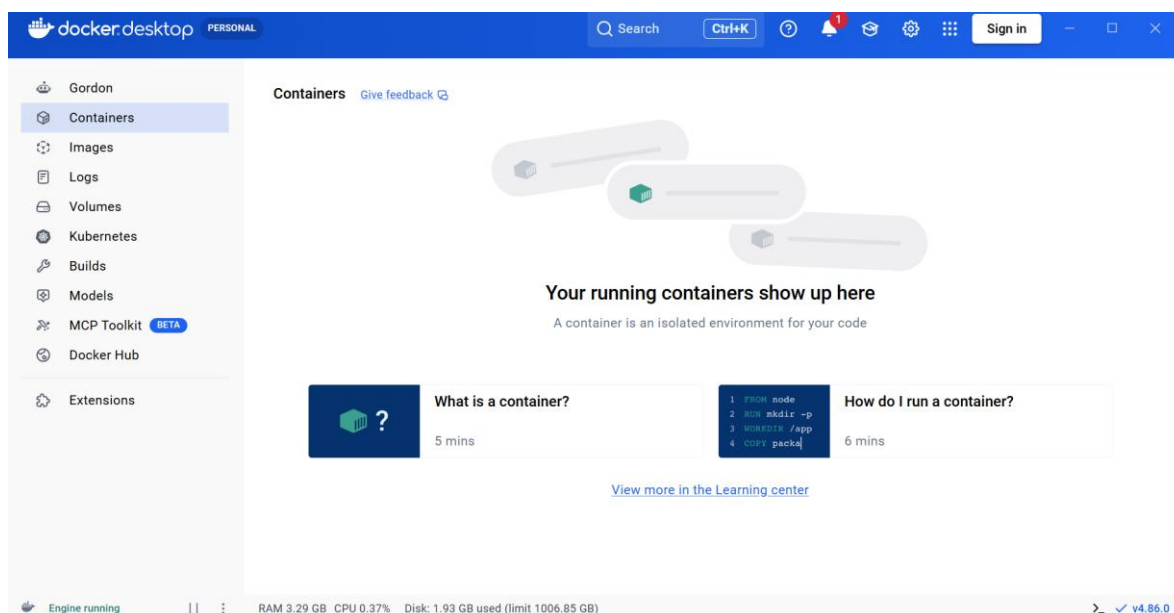
DOCENTE:

José Ubaldo Carvajal

Universidad de Manizales
Facultad de Ciencias e Ingeniería
Programa de Ingeniería
Manizales, Colombia

Descripción de la Aplicación Utilizada

Para el desarrollo de este análisis de seguridad, se utilizó **OWASP Juice Shop**, una de las aplicaciones web más modernas, sofisticadas y deliberadamente inseguras provistas por la comunidad de seguridad. Juice Shop está escrita completamente en JavaScript, utilizando tecnologías modernas de desarrollo de software como Node.js, Express y SQLite en el backend, y un frontend dinámico implementado en Angular. El propósito de esta plataforma es simular una tienda de comercio electrónico completamente funcional (compras, opiniones, búsquedas, pagos), pero plagada con vulnerabilidades reales que forman parte del estándar del OWASP Top 10, lo que la convierte en el entorno ideal para el entrenamiento, la auditoría de código y la práctica de metodologías de hacking ético.



```
Administrador: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> docker pull bkimminich/juice-shop
Using default tag: latest
latest: Pulling from bkimminich/juice-shop
3214acf345c0: Pull complete
dd64bf2dd177: Pull complete
99515e7b4d35: Pull complete
dd0d9bfd3bee: Pull complete
ebddc55facdc: Pull complete
99ba982a9142: Pull complete
bdfd7f7e5bf6: Pull complete
467afe997cab: Pull complete
2d4d7adf6272: Pull complete
d6b1b89eccac: Pull complete
2780920e5dbf: Pull complete
37c0829f1adc: Pull complete
1eed391ea893: Pull complete
7c12895b777b: Pull complete
c172f21841df: Pull complete
52630fc75a18: Pull complete
cac2ae0193cb: Pull complete
8224d91da70b: Pull complete
bd8962e29291: Pull complete
1a4be5562d92: Pull complete
2ba730cb9f70: Pull complete
cf397222899a: Pull complete
7cc70dfd88cf: Pull complete

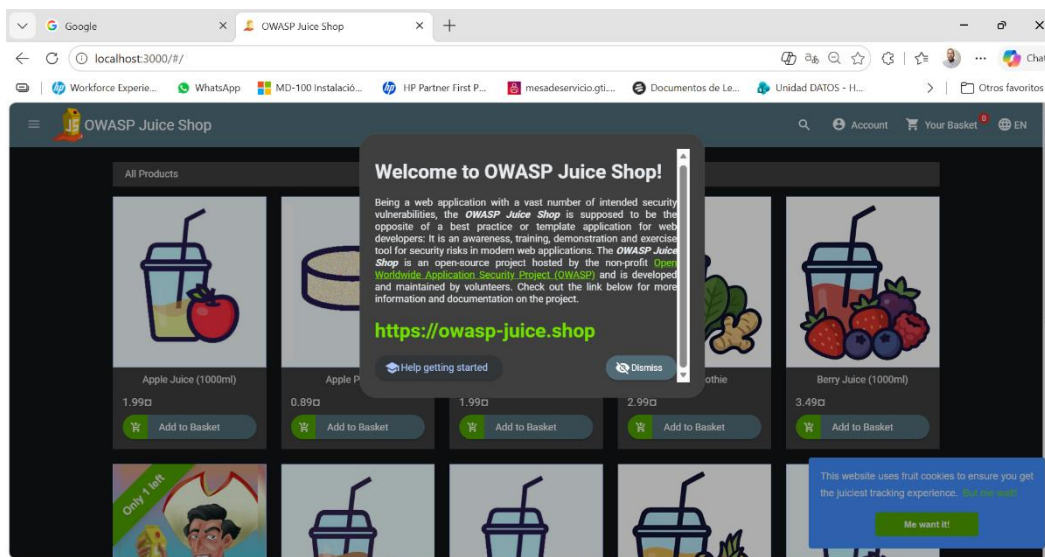
Engine running | RAM 3.29 GB CPU 0.37% Disk: 1.93 GB used (limit 1006.85 GB) v4.86.0
```

```
Administrador: Windows PowerShell

c172f21841df: Pull complete
52630fc75a18: Pull complete
cac2ae0193cb: Pull complete
8224d91da70b: Pull complete
bd8962e29291: Pull complete
1a4be5562d92: Pull complete
2ba730cb9f70: Pull complete
cf397222899a: Pull complete
7cc70dfd88cf: Pull complete
b839dfae01f6: Pull complete
3d9dea13d192: Download complete
Digest: sha256:73c53fbf442e8337b3ea3d98c7e8550308854701ebdfce4cc39768f36b75430e
Status: Downloaded newer image for bkimminich/juice-shop:latest
docker.io/bkimminich/juice-shop:latest
PS C:\WINDOWS\system32> docker run --rm -p 3000:3000 bkimminich

2ba730cb9f70: Pull complete
cf397222899a: Pull complete
7cc70dfd88cf: Pull complete
```

```
Administrador: Windows PowerShell
PS C:\WINDOWS\system32> docker run --rm -p 3000:3000 bkimminich/juice-shop
info: Detected Node.js version v24.19.0 (SUCCESS)
info: Detected OS linux (SUCCESS)
info: Detected CPU x64 (SUCCESS)
info: Configuration default validated (SUCCESS)
info: Entity models 21 of 21 are initialized (SUCCESS)
info: All dependencies in ./package.json are satisfied (SUCCESS)
info: Required file server.js is present (SUCCESS)
info: Required file index.html is present (SUCCESS)
info: Required file polyfills.js is present (SUCCESS)
info: Required file main.js is present (SUCCESS)
info: Required file styles.css is present (SUCCESS)
info: Required file matching /^hacking-instructor-.+\.js$/ is present (SUCCESS)
info: Port 3000 is available (SUCCESS)
info: Domain https://www.alchemy.com/ is reachable (SUCCESS)
warn: Environment variable ALCHEMY_API_KEY is not present (WARNING)
warn: "Mint the Honey Pot" challenge will not work as intended without a valid ALCHEMY_API_KEY
warn: "Wallet Depletion" challenge will not work as intended without a valid ALCHEMY_API_KEY
info: Check https://howto-web3.owasp-juice.shop for instructions on how to set up and configure the Alchemy API
warn: Domain http://localhost:11434/v1 is not reachable (WARNING)
warn: "Chatbot Prompt Injection" challenge will not work as intended without access to http://localhost:11434/v1
warn: "Greedy Chatbot Manipulation" challenge will not work as intended without access to http://localhost:11434/v1
warn: "AI Debugging" challenge will not work as intended without access to http://localhost:11434/v1
warn: "System Prompt Extraction" challenge will not work as intended without access to http://localhost:11434/v1
info: Check https://howto-llm.owasp-juice.shop for instructions on how to set up and configure the LLM API
info: Server listening on port 3000
```



2. Análisis de Vulnerabilidades (Categorización OWASP)

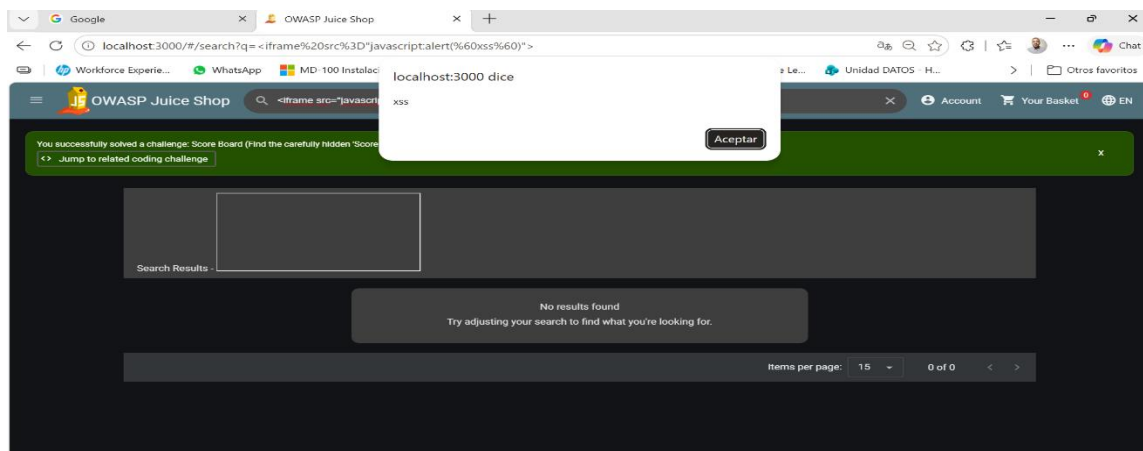
2.1. Vulnerabilidad 1: Cross-Site Scripting basado en DOM (DOM XSS)

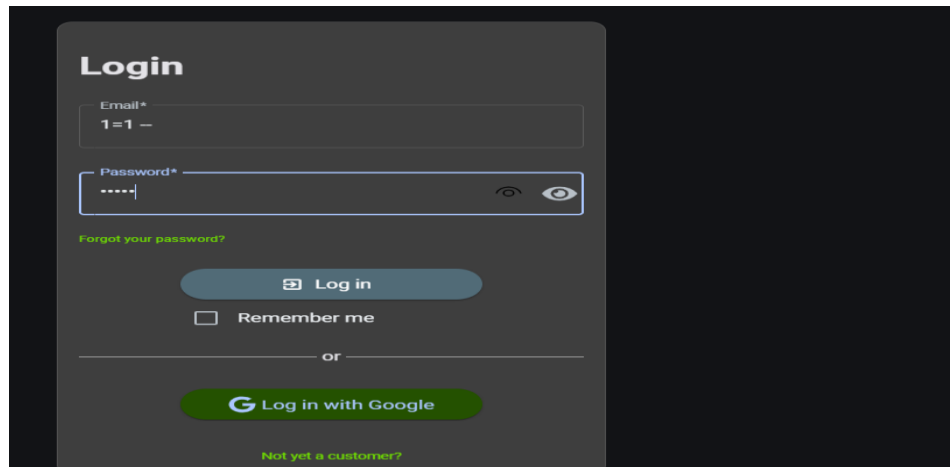
Categoría OWASP Top 10: A03:2021-Injection (Específicamente inyección del lado del cliente / Cross-Site Scripting)

Componente Afectado: Barra de búsqueda principal de la aplicación (/corporate-search / parámetro q)

Payload Utilizado: `<iframe src="javascript:alert('xss')">`

- **Procedimiento Ejecutado:** Se ingresó a la interfaz principal de OWASP Juice Shop. En el cuadro de texto de búsquedas, se introdujo el payload de inyección HTML/JavaScript interactivo. Al presionar Enter, la aplicación tomó
- la cadena ingresada desde la URL (`/#/search?q=...`) y la renderizó directamente en el árbol DOM de la página sin codificar ni filtrar los caracteres especiales (`<`, `>`, `'`). El resultado obtenido fue que el navegador web interpretó de forma nativa la etiqueta inyectada como código ejecutable del sitio, disparando inmediatamente una función de alerta con el texto "xss". Esto demuestra de forma irrefutable la existencia de una vulnerabilidad crítica de Cross-Site Scripting (XSS) en el lado del cliente.





Explicación Técnica y Causa Raíz (XSS)

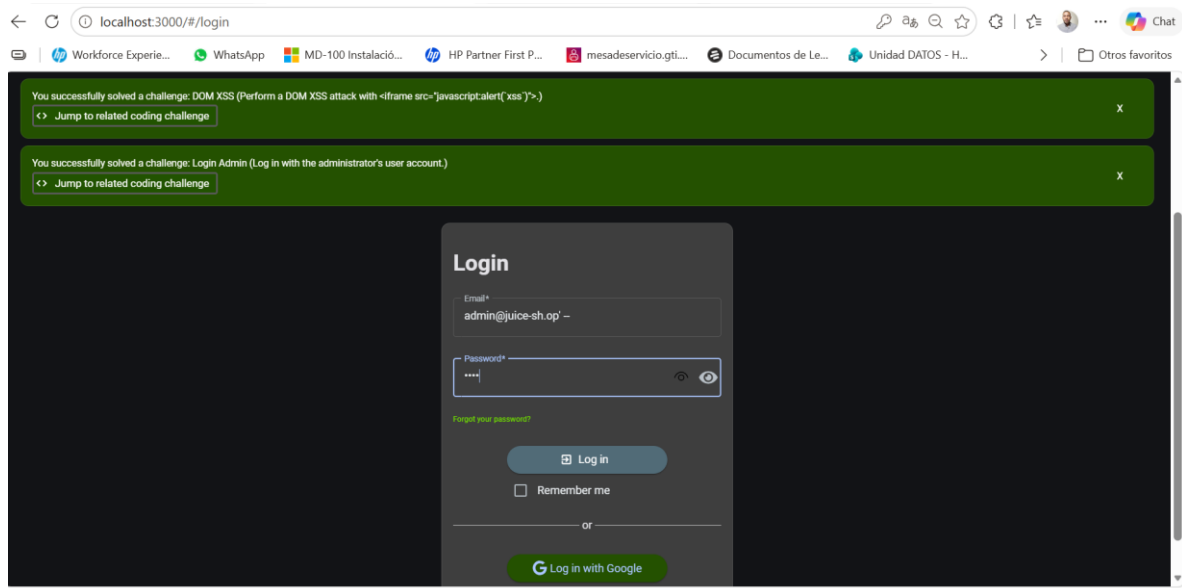
Esta vulnerabilidad ocurre en el frontend debido a una transferencia insegura de datos suministrados por el usuario hacia sumideros de ejecución de código (execution sinks) del DOM. Cuando el motor de Angular procesa el parámetro 'q' de la URL y lo asigna directamente a la propiedad de representación visual (como innerHTML o manipulación directa de nodos DOM) sin un mecanismo de saneamiento contextual previo, los datos del usuario rompen los límites sintácticos del HTML nativo. El navegador interpreta que los caracteres '<' y '>' corresponden a un nuevo elemento que debe ejecutarse dentro del contexto de seguridad del sitio.

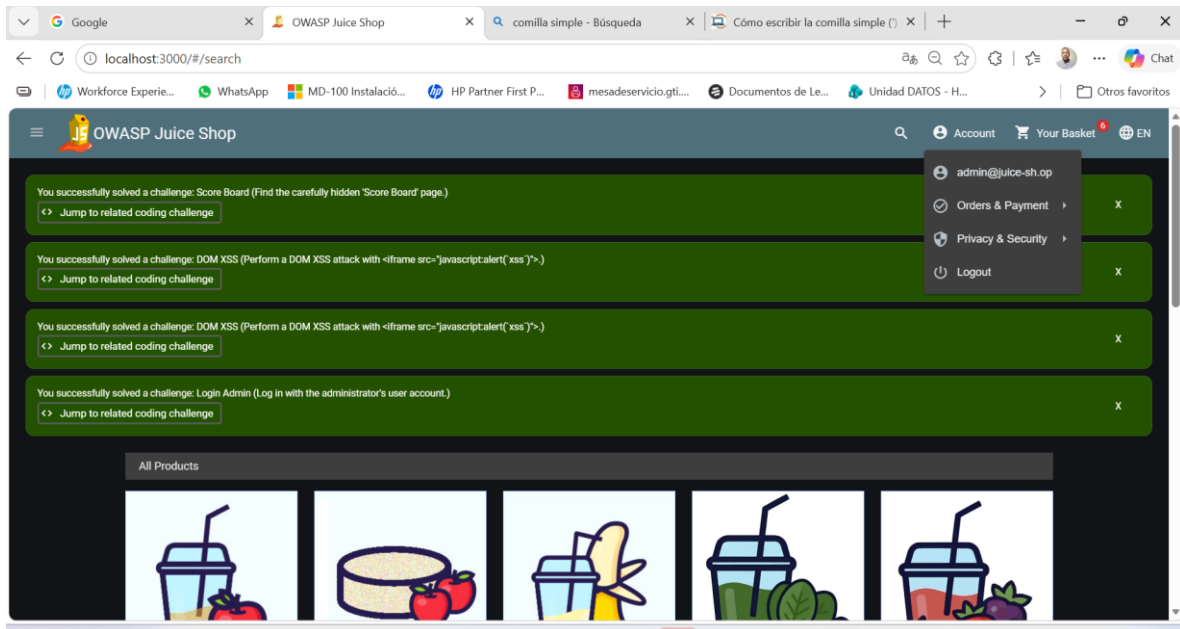
2.2. Vulnerabilidad 2: Inyección SQL (SQLi) en Autenticación

Categoría OWASP **Top 10:** A03:2021-Injection
Componente Afectado: Formulario de inicio de sesión (Login) / Parámetro Email
Payload Utilizado: *admin@juice-sh.op' --*

Procedimiento de Explotación: Se navegó hacia el formulario de inicio de sesión (Login) de OWASP Juice Shop. En el campo de correo electrónico, se introdujo el

vector de ataque (payload) mencionado anteriormente. Al enviar la solicitud, la comilla simple (') cerró prematuramente la validación del usuario en la base de datos, y los dos guiones (--) actuaron como un operador de comentario, instruyendo a la base de datos SQLite a ignorar por completo el resto de la consulta original (donde se verificaba si la contraseña era correcta). Como resultado, el servidor devolvió una sesión válida, permitiendo el acceso total con privilegios de Administrador sin conocer su contraseña legítima.





Explicación Técnica y Causa Raíz (SQLi)

Esta vulnerabilidad ocurre en el backend de la aplicación debido a una falta de sanitización y parametrización en las consultas enviadas a la base de datos. El sistema concatena directamente el texto que introduce el usuario (el campo Email) dentro de la estructura de la sentencia SQL de autenticación. Al ingresar caracteres de control como la comilla simple ('), el atacante logra romper la lógica original de la consulta, permitiendo inyectar comandos propios y alterar el comportamiento del sistema de bases de datos relacionales subyacente (SQLite).

2.3. Vulnerabilidad 3: Pérdida de Control de Acceso (Broken Access Control)

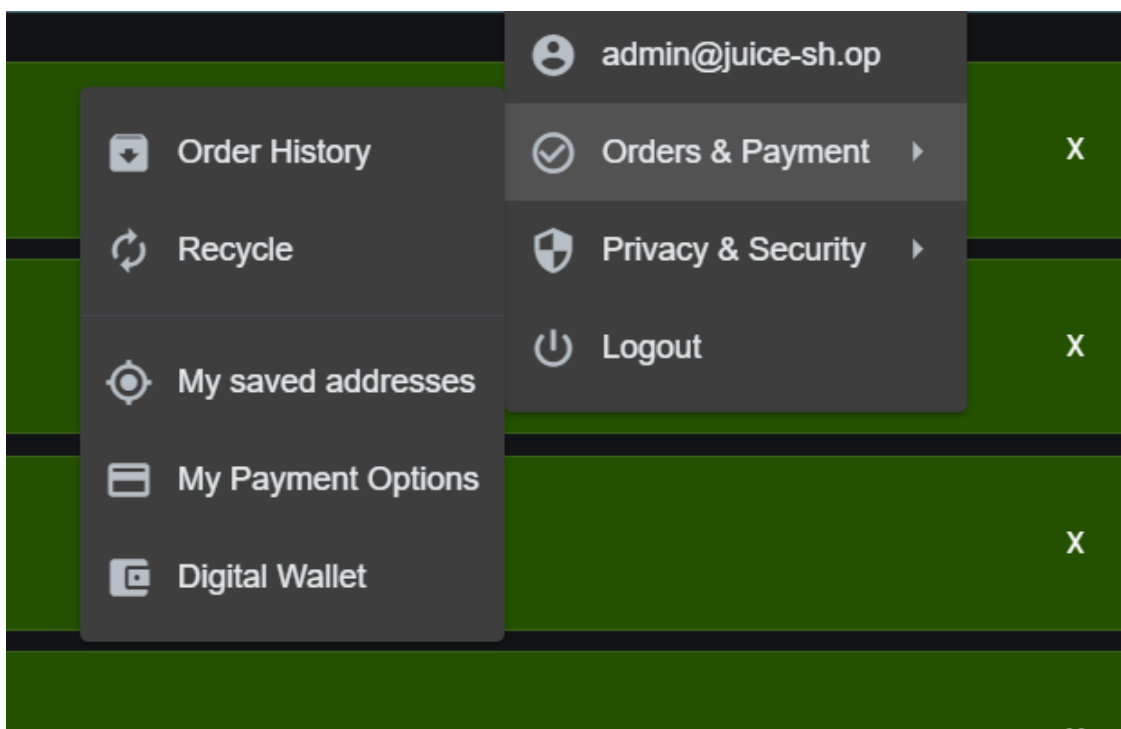
Categoría OWASP Top 10: A01:2021-Broken Access Control

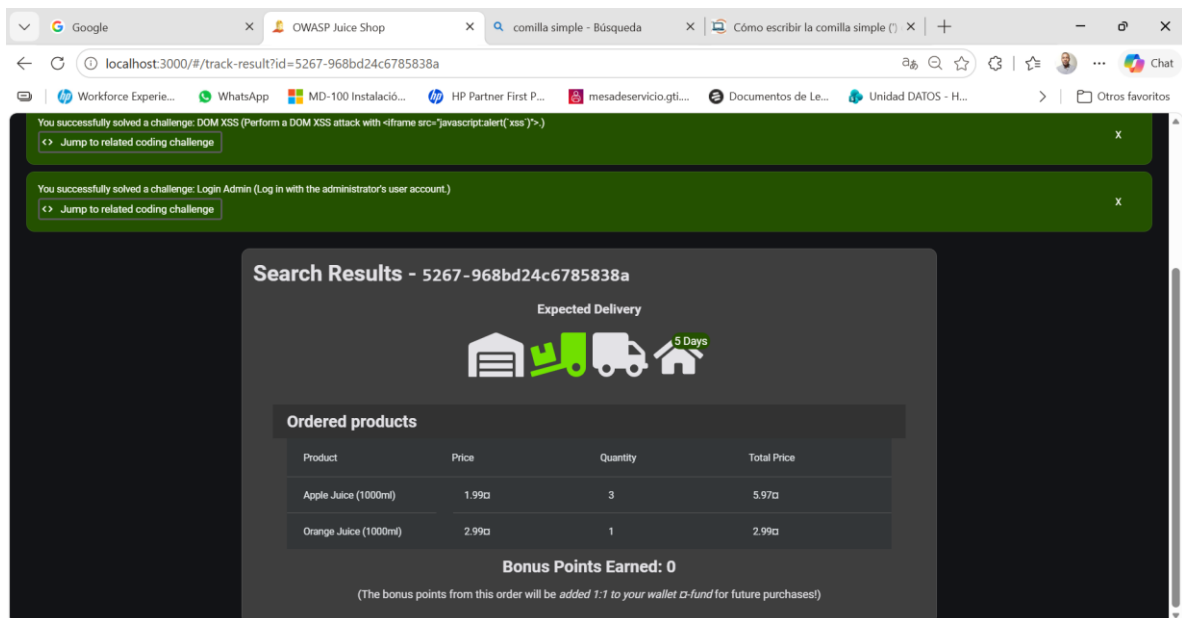
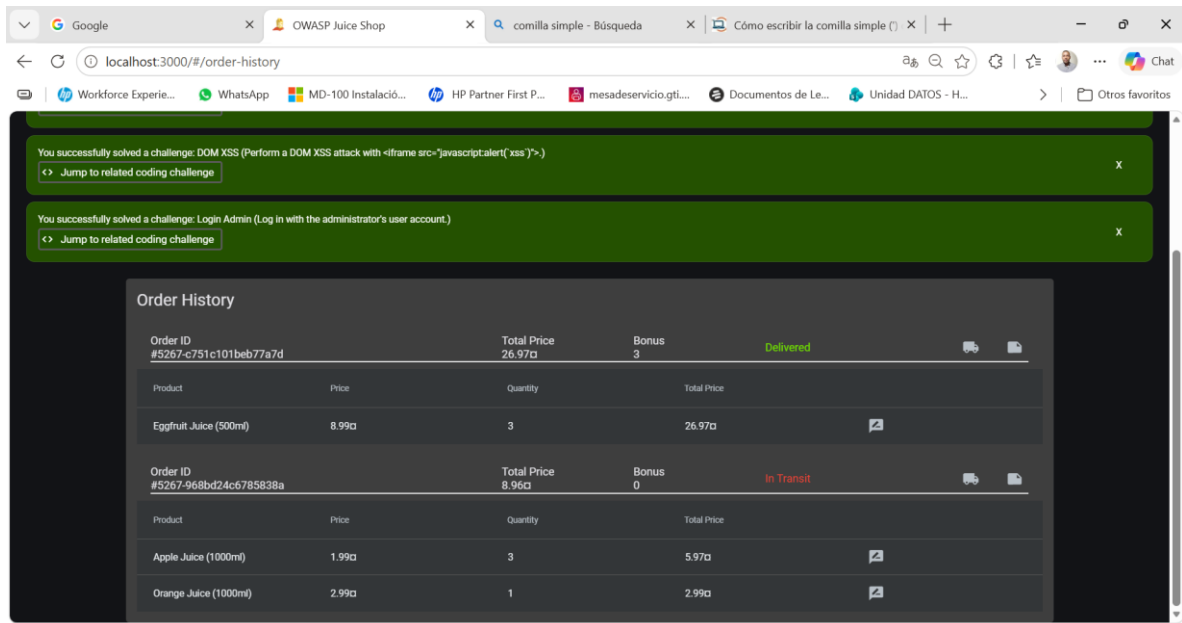
Componente Afectado: Ruta de rastreo de productos (/#/track-result?id=...)

Identificador Utilizado (ID): 5267-968bd24c6785838a

Procedimiento de Explotación (Evidencia): Se manipuló el parámetro de consulta en la barra de direcciones de la ruta de rastreo de productos (/#/track-result?id=...).

Al ingresar un identificador de seguimiento específico perteneciente a otra orden de compra externa (5267-968bd24c6785838a), la aplicación expuso abiertamente en pantalla el estado del pedido, los productos comprados (Apple Juice y Orange Juice) y los datos de entrega de un tercero, sin validar si el usuario en sesión era el dueño real de dicha compra.





Explicación Técnica y Causa Raíz (BAC)

Esta vulnerabilidad se presenta cuando el servidor web no realiza verificaciones de autorización adecuadas en el backend antes de mostrar un recurso protegido o confidencial. El sistema asume erróneamente que si un cliente cuenta con la URL o

el identificador específico de un recurso (Id), está autorizado de forma legítima para visualizarlo, lo que rompe la privacidad entre cuentas de usuarios al carecer de un mapa de control que asocie la identidad en sesión con la propiedad de los registros solicitados.

3. Recomendaciones de Mitigación (Normativa OWASP)

Para corregir y prevenir de raíz las vulnerabilidades explotadas en los tres ejercicios, se deben aplicar las siguientes medidas de seguridad en el desarrollo de la aplicación:

3.1. Mitigación para el Ejercicio 1: Cross-Site Scripting (XSS)

- Codificación de salida (Context-Aware Output Encoding): Convertir obligatoriamente todos los caracteres especiales introducidos en el buscador en sus entidades HTML seguras equivalentes antes de pintarlos en pantalla (por ejemplo, transformar < en < y > en >). Esto fuerza al navegador a interpretar la entrada estrictamente como texto y no como código ejecutable.
- Política de Seguridad de Contenido (CSP - Content Security Policy): Implementar encabezados HTTP CSP estrictos en el servidor. Esto restringe las fuentes desde las cuales se pueden cargar y ejecutar scripts en la página, bloqueando por completo la ejecución de scripts maliciosos en línea (inline scripts).

3.2. Mitigación para el Ejercicio 2: Inyección SQL (SQLi)

- Consultas Preparadas (Prepared Statements): Utilizar consultas parametrizadas de forma obligatoria a través del backend o del ORM. Esto garantiza que el motor de la base de datos (SQLite) trate los datos ingresados en el formulario de Login estrictamente como parámetros de texto, impidiendo que caracteres como la comilla simple (') o los guiones (--) alteren la estructura lógica de la consulta.

- Principio de Menor Privilegio: Configurar la cuenta de conexión de la aplicación a la base de datos con los permisos mínimos necesarios, evitando que un usuario comprometido pueda realizar consultas administrativas globales o escalamiento de privilegios.

3.3. Mitigación para el Ejercicio 3: Pérdida de Control de Acceso (Broken Access Control)

- Validación de Autorización en el Servidor: Implementar una verificación estricta en el backend para cada solicitud de rastreo o factura. El servidor debe comprobar activamente que el identificador del usuario que tiene la sesión iniciada coincida exactamente con el propietario legítimo del ID del pedido solicitado en la URL (id=...).
- Uso de Identificadores No Predecibles (UUID): Reemplazar los identificadores secuenciales simples por identificadores únicos universales aleatorios largos. Sin embargo, esto debe complementarse siempre con la validación en el servidor, ya que el UUID por sí solo no previene el acceso si la URL llega a filtrarse o ser descubierta.

4. Conclusiones

Evaluación del Riesgo en Entornos Modernos: El desarrollo de estos tres ejercicios prácticos demuestra que, a pesar de utilizar tecnologías modernas en el frontend y backend (como Angular y Node.js), las aplicaciones siguen siendo altamente susceptibles a fallos tradicionales del OWASP Top 10 si no se implementan controles de seguridad específicos desde el código de la aplicación.

La Importancia del Control de Entradas y Salidas: La raíz de las vulnerabilidades más críticas (como el XSS y la Inyección SQL) se reduce a una falta de validación y neutralización de los datos provistos por el usuario. Separar estrictamente los datos del usuario de los comandos de control del sistema es el pilar fundamental para mitigar riesgos de inyección en cualquier capa de la aplicación.

Foco en la Lógica de Negocio y Autorización: El ejercicio de Control de Acceso evidencia que la seguridad perimetral (como el inicio de sesión) no es suficiente si la aplicación no valida los permisos de forma continua en cada transacción. La seguridad efectiva requiere un enfoque de Defensa en Profundidad, protegiendo tanto la integridad del código como las reglas lógicas del negocio en el servidor.